ПРОЕКТ «АНТИФРОД» · НЕДЕЛЯ 6 · КОМАНДА ИЗ ТРЁХ УЧАСТНИЦ

Прескоринг на Airflow

Вся отложенная выборка оценивается пакетом по команде оркестратора, результаты ложатся в PostgreSQL, а компактная языковая модель пишет по ним отчёт для руководства — в деньгах, а не в метриках.

A · Назлы

слайды 1–3 · демо 1–3

зачем пакет, схема, DAG

B · Лия

слайды 4–5 · демо 4–6

скоринг в DAG и деньги

C · Шырайлым

слайды 6–7 · демо 7–9

отчёт, проверка, отказы

38 981

операция оценена одним запуском DAG за ≈ 40 секунд

71 313 USD

чистый эффект: предотвращено минус затраты на проверку

0

чисел в отчёте, которых нет в расчётах: проверяет код

Зачем пакетный режим и что добавилось

Онлайн остался прежним: операция по одной. Прескоринг — весь период сразу, по команде

	Онлайн (недели 3–5)	Прескоринг (неделя 6)
Когда	в момент операции	по команде или расписанию
Сколько	одна операция	вся выборка: 38 981
Кто зовёт	оператор, кнопка	Airflow, DAG
Ответ	за 0,15 с	весь пакет за ≈ 40 с
Кому	оператору: блокировать?	аналитике и руководству

Один и тот же код признаков

Airflow вызывает score_df из score.py — функцию пакетного скоринга недель 1–2. Новой реализации признаков не появилось: онлайн-сервис и прескоринг считают одинаково, поэтому их результаты сравнимы.

Сеть docker compose: 7 контейнеров

frontend :8501 · dashboard :8502 · backend :8000

онлайн-контур недель 3–5 — без изменений

▼ SQL ▼

db · PostgreSQL 16

+ 5 таблиц: holdout_transactions, scored_transactions, prescoring_runs, business_reports, llm_attempts

▲ чтение выборки · запись результатов ▲

airflow :8080

DAG prescoring, модель и score.py внутри; Connections и Variables

ollama

qwen2.5:3b — только текст отчёта по готовым числам

Adminer переехал на :8081 — порт 8080 занял Airflow

docker-compose.yml · airflow/Dockerfile · score.py (score_df)

DAG prescoring: девять задач

Ручной запуск; синие — прескоринг, зелёные — бизнес-отчёт

`create_tables` → `load_holdout` → `extract` → `score` → `load` → `summarize` → `build_facts` → `generate_report` → `check_report`

Задача	Что делает
`create_tables`	CREATE TABLE IF NOT EXISTS: init.sql сработал бы только на пустом томе
`load_holdout`	грузит data/holdout.csv, если таблица пуста — повторно не задваивает
`extract`	COPY выборки во временный CSV; в XCom уходит только путь
`score`	отбрасывает is_fraud, зовёт score_df для версий v2 и v1
`load`	DELETE прежних оценок версии → COPY новых, одной транзакцией
`summarize`	сверяет: оценено = объём выборки; пишет prescoring_runs
`build_facts`	SQL-агрегаты → финансовые показатели → строки facts
`generate_report`	промпт → Ollama, до 3 попыток, все в llm_attempts
`check_report`	сверяет числа, пишет business_reports в любом случае

Airflow, вкладка Grid: четыре запуска подряд



Идемпотентность

load удаляет оценки этой версии перед записью → повторный запуск не плодит дубликаты

XCom — только пути и счётчики. is_fraud удаляется до скоринга: утечки нет.

airflow/dags/prescoring.py · airflow/dags/sql/tables.sql

Модель внутри Airflow: главная сложность — версии

Артефакт модели читается только теми версиями библиотек, которыми был записан

Airflow 2.10 требует pandas < 2.2

Модель обучена на pandas 3.0.6, numpy 2.5.3, scikit-learn 1.9.1, Python 3.13

В одном окружении они не уживаются

▼ решение ▼

Окружение Airflow

Python 3.12, сам Airflow и провайдер PostgreSQL

model-venv

Python 3.13 + версии обучения (requirements-model.txt)

```
@task.external_python(
    python="/opt/airflow/model-venv/bin/python")
def score(job): ...   # задача идёт отдельным
                      # процессом в своём venv
```

Грабли	Как обошли
can't adapt numpy.int64	запись через COPY из CSV, а не insert_rows
DAG не виден в интерфейсе	ошибка импорта → Import Errors, dags list-import-errors
localhost вместо имени	в сети Docker база — db, модель — ollama
Ollama отвечает 404	модель не скачана: ollama pull; DAG качает её сам
Таймаут генерации	timeout 900 с; Docker Desktop — от 8 ГБ памяти
Пропали временные файлы	факты дублируются в prescoring_runs.facts

Проверили на деле

в задаче score печатается «колонка is_fraud удалена до скоринга: True», а summarize падает, если оценено не столько строк, сколько в выборке.

airflow/Dockerfile · airflow/requirements-model.txt

Метрики модели → деньги

Считает SQL и Python; допущения вынесены в Airflow Variables

Показатель	Как считается	Выборка 38 981
Предотвращено	сумма пойманного фрода (TP)	113 оп. · 71 541 USD
Понесено	сумма пропущенного фрода (FN)	18 оп. · 3 022 USD
Ложные блокировки	сумма честных, помеченных (FP)	1 оп. · 23,49 USD
Затраты на проверку	(TP + FP) × 2 USD	114 × 2 = 228 USD
Упущенная комиссия	сумма FP × 1,5%	0,35 USD
Чистый эффект	предотвращено − затраты − комиссия	71 313 USD
Покрытие фрода	предотвращено / весь фрод выборки	96,0%

Сравнение версий (доп.): прежняя модель v1 предотвратила бы 70 663 USD, чистый эффект 70 437 USD — на 876 USD меньше. Обе версии скорятся в одном запуске и сравниваются в одном отчёте.

Откуда взяты допущения

Проверка 2 USD — 4–5 минут аналитика при ставке ≈ 25–30 USD в час.
Комиссия 1,5% — типичная межбанковская ставка по картам в США: её банк теряет, когда блокирует честную покупку.

Меняются без правки кода

Admin → Variables: review_cost, commission_rate, threshold, llm_model

Если удвоить обе ставки

чистый эффект 71 085 USD вместо 71 313 — минус 0,3%: срабатываний мало, вывод не меняется

airflow/dags/report_lib.py (compute_finance, build_facts) · prescoring_runs

Отчёт пишет модель, числа считает код

Языковая модель получает готовые строки и обязана переписать их символ в символ

```
facts = {
  "операций": "38 981",
  "предотвращённые_потери":
      "113 операций на сумму 71 541 USD",
  "чистый_эффект": "71 313 USD", ... }
```

▼ промпт: только эти значения, ничего не считать ▼

```
allowed = числа из facts
extra   = числа из отчёта − allowed
check_passed = not extra
```

Лишнее число → повтор с подсказкой, до 3 попыток. temperature 0.2 и фиксированный seed: ответ воспроизводится.

Чего не ловит: перепутанный смысл при верных числах. Поэтому отчёт читает человек.

Панель мониторинга, блок «Прескоринг (Airflow)»

Предотвращено, USD	Пропущено, USD	Чистый эффект, USD	Покрытие фрода
71 541	3 022	71 313	96,0%
пойманный фрод	понесённые потери	после затрат на проверку	модель v2, порог 0.5

Проверка чисел пройдена · mock · попыток: 2 · запуск manual__2026-11-12T00:00:00+00:00

За период 02.12.2020 – 11.12.2020 обработано 38 981 операций, на проверку направлено 114. Предотвращены потери: 113 операций на сумму 71 541 USD, покрытие мошенничества 96,0%. Понесённые потери: 18 операций на сумму 3 022 USD. Ложные блокировки: 1 операция на сумму 23,49 USD; операционные затраты 228 USD, упущенный доход 0,35 USD. Чистый эффект 71 313 USD. Вопрос: приемлем ли текущий уровень ложных блокировок?

Что здесь чьё

Плитки и суммы	посчитаны SQL-запросом к таблицам прескоринга
Зелёная строка	результат автоматической проверки чисел, модель и число попыток
Текст ниже	единственное, что написала языковая модель

report_lib.py (make_prompt, check_report) · tests/test_report.py — 6 тестов

Отказ компонента и план демонстрации

Ситуация	Что происходит
Повторный запуск DAG	строк столько же — дубликатов нет
stop ollama → запуск	падает только generate_report, после 2 повторов
	скоринг, prescoring_runs и факты уже сохранены
start ollama → Clear	перезапускаются две задачи, не весь DAG
Сменили допущение	следующий запуск — новые числа в отчёте

Почему это важно

прескоринг — ночной пакет: упавший отчёт не должен требовать повторного скоринга 39 тысяч операций.

Не сделали

Alembic, расписание и Datasets: выборка статическая, запуск ручной, схема одна

Демонстрация · 10 минут

1	A	up -d → ps: 7 сервисов, db/backend/airflow healthy
2	A	SELECT count(*) FROM holdout_transactions → 38 981
3	A	запуск в Airflow: 9 задач зелёные, лог score
4	B	count(scored_transactions) = 38 981, запись о запуске
5	B	запускаем ещё раз: число строк не изменилось
6	B	review_cost = 4, запуск: затраты 456 USD
7	C	отчёт и «проверка пройдена»; то же внизу панели
8	C	число из отчёта подтверждаем SQL-запросом
9	C	stop ollama, запуск, start ollama, Clear

zapusk_dag.bat · llm_analiz.bat · docs/weeks/неделя6.md